科目：データベースの基礎

第5回 SQL

講師：鈴木源吾

講義ルールの変更：小テストレポートの締切後の提出

- 小テストが7点未満（または欠席）の人は，レポートを提出する（満点7点）．これは変わらない
- **変更**：締切後に「【締切後】小テストN レポート」から提出した場合は，**1点減の満点6点**で評価する
  - 締切どおりに出せば7点．締切を過ぎても 2/19 18:30 までは提出でき，6点（小テスト満点の6割）は確保できる
- 小テスト1 レポートの締切：**11/11 講義開始30分後**（返却は 11/4）
- 学生ポータルの講義ルール（PDF）を更新済み

前回の復習

1. リレーショナルデータモデルの操作：リレーショナル代数
    - 集合演算（和・差・共通・直積）と固有の演算（射影 π ・選択 σ ・結合 $\bowtie$ ・商）
    - 結果もリレーション（集合）．射影では重複がまとまる．内側の演算から順に適用する
2. SQLについて：背景・歴史・利用法など
    - 非手続き型（宣言的）．DDL・DML・DCL
3. SQLの基本：SELECT文による検索
    - SELECT句（射影）・FROM句・WHERE句（選択）．文字列は '…' で囲む

第4回演習の解説

- 第4回ポイント資料の裏面（演習1・演習2・確認問題）の正解を解説する
- 解説は，演習の締切（講義開始30分後）のあとに行う
- 解答例は電子的には配布しない．**自分の解答に転記すること**（解説中の撮影は不可）
  - 講義中に自分の解答へ書き足す・訂正するのはよい．そのまま試験に持ち込める
- 提出がまだの人は，WebClass の「【締切後】第4回 演習」から提出する（0.7点で評価）

今回のトピック

0. 実習環境の確認（今回は SA なし．未解決の人は Colab で進める）
1. SELECT文：前回の続き（複雑な条件・複数テーブルの結合・並べ替え・集約）
2. SQLとリレーショナル代数の対応
3. （参考）テーブルとデータの作成・削除・更新 ← 小テストの範囲外
4. 今回の演習と小テストの「型」
5. **小テスト1（講義の最後15分）**：範囲は第1〜3回．持ち込みは第1〜3回のポイント資料のみ

教科書：第5章 p.48〜62

実習環境の最終確認

- 前回の確認問題1（`SELECT version();`）が空欄だった人 → 今回は SA がいないので，**講義の前後で教員に相談**．演習は Colab で進めてよい（SQL は同じ）
- サンプルデータが入っているか：pgAdmin のクエリツールで次を実行

```
SELECT COUNT(*) FROM 従業員;    -- 4
SELECT COUNT(*) FROM track;     -- 3503
```

- 入っていなければ，学生ポータル 第4回ページの `sample.sql` → `chinook_pg.sql` の順に開いて実行（F5）
- Colab の人：第5回ページのノートブックを上から実行（sample と Chinook が自動で入る）

Topic 1

SELECT文：前回の続き

トピックのキーワード

- AND・OR，IS NULL，LIKE
- 複数テーブルの結合：FROM 複数＋WHERE，JOIN … ON
- ORDER BY，集約関数（COUNT・SUM），GROUP BY

SELECT文の例：少し複雑な条件指定

WHERE句の条件指定には，AND，ORを利用できる

```
SELECT * FROM 従業員 WHERE 住所 = '横浜市' AND 職級 = 2;
```

- テーブル 従業員 から，住所が「横浜市」に等しく，**かつ**，職級が 2 に等しいタプルの，すべての属性を検索する
- 条件式で使えるもの
  - AND：かつ，OR：または，NOT：でない
  - カッコを使い，AND・ORの優先順位を指定できる

従業員番号	部門番号	氏名	住所	職級
1	1	佐藤一郎	横浜市	2
3	2	鈴木三郎	横浜市	2

SELECT文の例：NULLの条件指定

- 属性の値がNULLであるかは，「IS NULL」で判定する
  - 例） `WHERE 住所 IS NULL` ：住所の登録がない行だけを選ぶ
- NULLでないことを調べるには，「IS NOT NULL」を使う
  - 例） `WHERE 電話番号 IS NOT NULL` ：電話番号が登録してある行だけを選ぶ
- **`= NULL` とは書けない**（第3回：空値は比較できない． `住所 = NULL` の結果は常に不明で，1行も選ばれない）

```
SELECT first_name, last_name FROM customer WHERE company IS NOT NULL;
```

- Chinook の customer で，会社名が登録されている顧客（10行）

SELECT文の例：二つのテーブルをまとめたい

例えば，次の結果を得るにはどうSQLを書けばよいのか？

「従業員の氏名と部門名の組み合わせをすべてほしい」

- テーブル 従業員 に，氏名の情報がある
- テーブル 部門 に，部門名の情報がある

→ テーブル 従業員 とテーブル 部門 の**二つのテーブルをまとめて検索**する必要がある

従業員

従業員番号	部門番号	氏名
1	1	佐藤一郎
2	1	田中次郎
3	2	鈴木三郎
4	2	山田四郎

部門

部門番号	部門名
1	営業部
2	開発部



欲しい結果

氏名	部門名
佐藤一郎	営業部
田中次郎	営業部
鈴木三郎	開発部
山田四郎	開発部

SELECT文の例：二つのテーブルをまとめたい

以下のSQL文で欲しい情報は検索できる

```
SELECT 氏名, 部門名 FROM 従業員, 部門
WHERE 従業員.部門番号 = 部門.部門番号;
```

- FROM句に複数のテーブルを指定する．結果は**直積**＝すべての組み合わせ（4×2＝8行）
- WHERE句の条件式（部門番号が等しい）で絞り込む → 4行
- 属性が含まれるテーブルを区別したいときは「.（ドット）」を使う
  - 属性名の衝突がおきなければテーブルを指定しなくてもよい

SELECT文の例：二つのテーブルをまとめたい（直積→選択→射影）

FROM句の直積（8行）から，WHERE句で部門番号が等しい行（枠）を選び，SELECT句で氏名と部門名を取り出す

従業員（2列のみ）

部門番号	氏名
1	佐藤一郎
1	田中次郎
2	鈴木三郎
2	山田四郎

部門

部門番号	部門名
1	営業部
2	開発部

→

直積（従業員×部門）

従業員.部門番号	氏名	部門.部門番号	部門名
1	佐藤一郎	1	営業部
1	佐藤一郎	2	開発部
1	田中次郎	1	営業部
1	田中次郎	2	開発部
2	鈴木三郎	1	営業部
2	鈴木三郎	2	開発部
2	山田四郎	1	営業部
2	山田四郎	2	開発部

→

部門番号が等しい行を選択し氏名と部門名で射影

氏名	部門名
佐藤一郎	営業部
田中次郎	営業部
鈴木三郎	開発部
山田四郎	開発部

SELECT文の例：JOIN句の使用

前ページでは，FROM句で二つのテーブルを指定し，WHERE句で絞り込んだ．SQLには複数のテーブルを結合する**JOIN句**があり，これを用いると以下のように書けて同じ結果が得られる（こちらの方がわかり易い．**本講義ではこちらを推奨**）

```
SELECT 氏名, 部門名 FROM 従業員
JOIN 部門
ON 従業員.部門番号 = 部門.部門番号;
```

- 結合したいテーブルを，JOIN句で指定
- 結合に用いる条件式を，ON句で指定（ふつうは **主キー ＝ 外部キー**）
- さらに条件を付けるときは，そのあとに WHERE句を書く
  - 例） `… ON 従業員.部門番号 = 部門.部門番号 WHERE 部門名 = '営業部';`

SELECT文の例：CROSS JOIN句

CROSS JOIN句を用いると直積を求めることができる．下図のように，FROM句に2つのテーブルを指定する場合と同じ結果が得られる

```
SELECT 氏名, 部門名 FROM 従業員
CROSS JOIN 部門;
```

	氏名	部門名
1	佐藤一郎	営業部
2	田中次郎	営業部
3	鈴木三郎	営業部
4	山田四郎	営業部
5	佐藤一郎	開発部
6	田中次郎	開発部
7	鈴木三郎	開発部
8	山田四郎	開発部

```
SELECT 氏名, 部門名
FROM 従業員, 部門;
```

	氏名	部門名
1	佐藤一郎	営業部
2	田中次郎	営業部
3	鈴木三郎	営業部
4	山田四郎	営業部
5	佐藤一郎	開発部
6	田中次郎	開発部
7	鈴木三郎	開発部
8	山田四郎	開発部

便利なSELECT文の機能：文字列部分一致．LIKE演算子

```
SELECT 氏名, 従業員番号 FROM 従業員
WHERE 氏名 LIKE '鈴木%';
```

- テーブル 従業員 から，「氏名」が「鈴木」で始まるタプルについて，氏名と従業員番号を検索する
- LIKE演算子の書き方
  - `%`：任意の長さの文字列（0文字でもよい）
  - `_`：任意の1文字
  - 例）`'%郎'`：「郎」で終わる　`'%田%'`：「田」を含む
- PostgreSQL では大文字・小文字を区別する（`'a%'` と `'A%'` は別）

便利なSELECT文の機能：結果の並べ替え．ORDER BY句

```
SELECT 職級, 氏名 FROM 従業員
ORDER BY 職級, 氏名;
```

- テーブル 従業員 から，職級と氏名を，最初に職級の順，同じ職級の場合にはさらに氏名で並べ替えて返却する
- ORDER BY句の書き方
  - 複数の属性名をカンマで区切って指定
  - 先に書いた属性を使って並べ替え，値が同じなら次の属性で並べ替え
  - 降順にするには属性名のあとに `DESC` （昇順は `ASC` ．省略時は昇順）
- ORDER BY がなければ，結果の順序は保証されない（リレーションは集合）

便利なSELECT文の機能：集約演算，行数のカウント

集約演算とは，複数の行の中の値を合計したり数えたりして一定の値を導き出す演算

```
SELECT COUNT(*) FROM 従業員;
```

- テーブル 従業員 から，見つかったタプル数（行数）を返却する → 4
- COUNT()：行を数える関数
  - SELECT句にこのような集計用の関数を書くことができる
  - ()の中には属性名を指定できる．ここではすべての属性を表す「*」を指定
  - **`COUNT(*)` はNULLを含んだ行も数えるが，`COUNT(属性名)` はその属性がNULLの行を数えない**
  - 例）Chinook の customer：`COUNT(*)` は 59，`COUNT(company)` は 10

便利なSELECT文の機能：合計・平均・最大・最小

```
SELECT SUM(数量) FROM 供給;
```

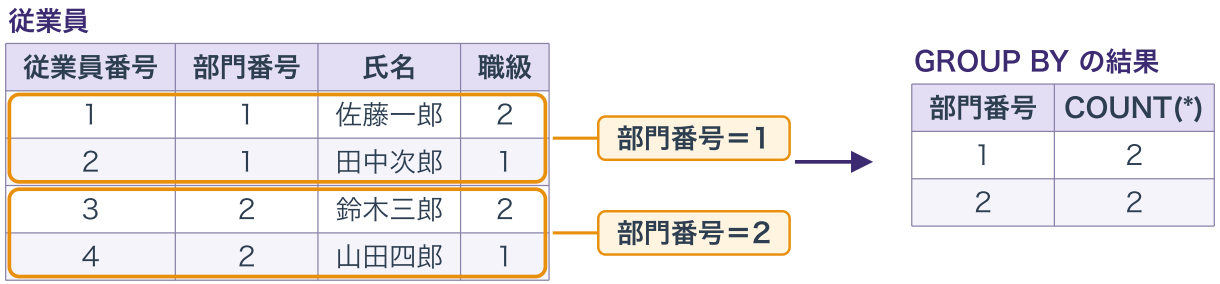
- テーブル 供給 から，属性 数量 の値の合計を返却する → 600
- 集約関数
  - `SUM(属性)`：合計　`AVG(属性)`：平均　`MAX(属性)`：最大　`MIN(属性)`：最小
  - `SELECT MAX(単価), MIN(単価) FROM 供給;` のように複数書ける
- 集約関数の結果は1行になる．集約していない属性を一緒に SELECT 句に書くことはできない（次のGROUP BYを使う）

グループ化と集約関数：GROUP BY句

属性の値によってグループ化し，グループごとに合計などを求められる

```
SELECT 部門番号, COUNT(*) FROM 従業員 GROUP BY 部門番号;
```

- テーブル 従業員 から，部門番号ごとにグループ化して，各グループの行数を返却する → (1, 2), (2, 2)
- GROUP BYの後には（複数の）属性を指定できる．属性の値（の組み合わせ）ごとにグループが作られ，集計が適用される
- SELECT句に書けるのは，グループ化した属性と集約関数だけ



Topic 2

SQLとリレーショナル代数

トピックのキーワード

- 射影 ↔ SELECT句，選択 ↔ WHERE句，結合 ↔ JOIN句
- UNION・EXCEPT・INTERSECT，DISTINCT

SQLとリレーショナル代数

- 前回，リレーショナルデータベースの基礎として，リレーショナル代数を学んだ
  - 射影，選択，結合・・・
- 前回と今回，リレーショナルデータベースの操作言語としてSQLのSELECT文を学んだ
  - SELECT ほげほげ FROM なんちゃら WHERE ...
- この2つの対応関係を理解しておこう（小テストの型H）

SQLとリレーショナル代数：射影と選択

- 射影はSELECT句に，選択はWHERE句に，それぞれ対応する
- $\pi_{\text{氏名}}(\sigma_{\text{職級}=1}(\text{従業員})) \leftrightarrow$ `SELECT 氏名 FROM 従業員 WHERE 職級 = 1;`
  - 式の内側（σ）がWHERE，外側（π）がSELECT

演算	意味	SQL	代数
射影	リレーションから特定の属性を抜き出す	`SELECT 氏名`	$\pi_{\text{氏名}}$
選択	リレーションから条件を満たすタプルを選ぶ	`WHERE 職級 = 1`	$\sigma_{\text{職級}=1}$
（対象）	演算を適用するリレーション	`FROM 従業員`	(従業員)

SQLとリレーショナル代数：結合と直積

- **結合**（2つのリレーションのタプルを，属性の値の関係でくっつける）は，JOIN句が直接対応している
  - 結合したいテーブルを JOIN句で，結合に用いる条件式を ON句で指定

```
SELECT 氏名, 部門名 FROM 従業員
JOIN 部門 ON 従業員.部門番号 = 部門.部門番号;
```

$\leftrightarrow \pi_{\text{氏名,部門名}}(\text{従業員} \bowtie_{\text{従業員.部門番号}=\text{部門.部門番号}} \text{部門})$

- **直積**は，CROSS JOIN句が直接対応している
  - `SELECT * FROM 従業員 CROSS JOIN 部門;` $\leftrightarrow \text{従業員} \times \text{部門}$

結合の「直積〜選択〜射影」による実現

FROM句（直積）・WHERE句（選択）・SELECT句（射影）の組み合わせによって，結合を実現できる

SQL	リレーショナル代数
`FROM 従業員, 部門`	直積 $従業員 \times 部門$
`WHERE 従業員.部門番号 = 部門.部門番号`	選択 $\sigma_{従業員.部門番号=部門.部門番号}$
`SELECT 氏名, 部門名`	射影 $\pi_{氏名,部門名}$

- $\pi_{氏名,部門名}(\sigma_{従業員.部門番号=部門.部門番号}(従業員 \times 部門))$ は，前ページの**結合**と同じ結果になる

SQLとリレーショナル代数：集合操作

和・差・共通集合については，それに対応する構文がある（2つの SELECT の結果の属性が同じ形であること＝和両立）

代数	SQL
和 $R \cup S$	`SELECT * FROM R UNION SELECT * FROM S;`
差 $R - S$	`SELECT * FROM R EXCEPT SELECT * FROM S;`
共通 $R \cap S$	`SELECT * FROM R INTERSECT SELECT * FROM S;`

- 例）`SELECT 住所 FROM 従業員 UNION SELECT 住所 FROM 業者;` → 横浜市，静岡市，東京都，アメリカ合衆国

代数とSQLの違い：重複

- リレーショナル代数の結果は**集合**：$\pi_{\text{部門番号}}(\text{従業員}) = \{(1), (2)\}$
- SQL の `SELECT 部門番号 FROM 従業員;` の結果は **4行**（1, 1, 2, 2）
  - SQL のテーブルと問合せの結果は，重複する行を許す（集合ではなく「バッグ」）
- 重複を除きたいときは **DISTINCT**

```
SELECT DISTINCT 部門番号 FROM 従業員;    -- 2行
```

- UNION・EXCEPT・INTERSECT は，集合演算なので重複を除く

Topic 3

（参考）テーブルとデータの作成・削除・更新

トピックのキーワード

- 参考扱い．小テストの範囲外（ポイント資料の 5. に要約）
- CREATE TABLE・ALTER TABLE・DROP TABLE，INSERT・UPDATE・DELETE，ビュー

データベースにおけるデータ型

- データベースに格納される値には「データ型」がある（第3回のドメイン制約）
- SQL（標準）が提供するデータ型の例
  - 数値：整数 `INTEGER`，`NUMERIC`．浮動小数点数 `REAL`，`FLOAT`
  - 文字列：`CHAR(n)` 固定長，`VARCHAR(n)` 最大 n の可変長，`TEXT` 可変長
  - 日付・時刻：`DATE` 年月日，`TIMESTAMP` 年月日時分秒
  - そのほか，ビット列，任意のバイト列，図形，JSON 等々
- DBMSが違うと提供されるデータ型の種類も異なる

PostgreSQLのデータ型

以下がPostgreSQLの「代表的な」データ型である

分類	データ型	大きさ	範囲など	値の例
数値：整数	`integer`	4バイト	-2147483648〜2147483647	100
数値：浮動小数点	`double precision`	8バイト	15桁精度	1.234
数値：任意精度	`numeric`	可変長（精度と桁を指定可）	小数点より上は131072桁，下は16383桁まで	1.234
数値：連番型	`serial`	4バイト	1〜2147483647．タプルの識別子を生成できる	100
文字：上限付き可変長	`varchar(n)`	n に文字数を指定		ABC
文字：固定長	`char(n)`	n に文字数を指定		ABC
文字：制限なし可変長	`text`	–		ABC
日付	`date`	4バイト	紀元前4713年〜5874897年	1999-01-08
時刻	`time`	8バイト	00:00:00〜24:00:00	04:05:06
タイムスタンプ	`timestamp`	8バイト	紀元前4713年〜294276年	1999-01-08 04:05:06

- ID は連番型 `serial`，整数は `integer`，小数は `numeric`，文字は `text`，日時は `timestamp`（参考：PostgreSQL マニュアル 第8章）

テーブルを作る：CREATE TABLE 文と制約

- 一般形：`CREATE TABLE テーブル名（カラム名 型 制約, ...);`
- 制約（第3回）：`PRIMARY KEY` 主キー（UNIQUE かつ NOT NULL），`UNIQUE`，`NOT NULL`，`CHECK（条件）`，`REFERENCES テーブル名(属性名)` 外部キー

```
CREATE TABLE 部門 (
    部門番号  INTEGER PRIMARY KEY,
    部門名    TEXT UNIQUE NOT NULL);
CREATE TABLE 従業員 (
    従業員番号  INTEGER PRIMARY KEY,
    部門番号  INTEGER NOT NULL REFERENCES 部門(部門番号),
    氏名  TEXT NOT NULL,
    住所  TEXT,
    職級  INTEGER CHECK (職級 >= 1));
```

- `sample.sql` の中身はこの形（第3回の Colab で制約違反を試したのも同じ）

テーブルの削除・定義の変更：DROP TABLE，ALTER TABLE

- `DROP TABLE テーブル名;`：テーブルを削除する（データも消える）
  - SQLでは，様々なものを削除するときに「DROP」を使う
- `ALTER TABLE テーブル名 変更の内容;`：テーブルの定義を変える

```
ALTER TABLE 従業員 ADD COLUMN 電話 TEXT;              -- 属性の追加
ALTER TABLE 従業員 ALTER COLUMN 住所 SET NOT NULL; -- 制約の追加
ALTER TABLE 従業員 DROP COLUMN 電話;                  -- 属性の削除
ALTER TABLE 部門 RENAME COLUMN 部門名 TO 名称;       -- 属性の名前変更
```

タプルの追加・更新・削除：INSERT，UPDATE，DELETE

```
INSERT INTO 部門 VALUES (3, '総務部');
INSERT INTO 部門 (部門番号, 部門名) VALUES (4, '人事部'), (5, '経理部');

UPDATE 従業員 SET 職級 = 職級 + 1 WHERE 従業員番号 = 1;

DELETE FROM 従業員 WHERE 氏名 = '佐藤一郎';
```

- INSERT：属性名の並びを省略すると，テーブル定義の順に値を指定する．複数行を一度に入れられる
- UPDATE：`SET 属性 = 値/式`．WHERE がないと**すべての行**が変わる
- DELETE：WHERE がないと**すべての行**が消える（テーブルは残る）
- 外部キー制約・NOT NULL 制約などに違反する更新はエラーになる（第3回）

ビュー（view）

- アプリケーションはテーブルの一部しか使わない → 最初からテーブルの一部だけを見せる「仮想的な」表を**ビュー**と呼ぶ
- ビューは「問合せ」を使って定義する．テーブルと同じように SELECT できる
  - ビューへの問合せは，ビュー定義の問合せと合成したものが実行される（代数の再帰性）
  - テーブルの形が変わってもビューを直せばアプリケーションは変更せずに済む

```
CREATE VIEW 営業部員 AS
  SELECT 氏名, 住所 FROM 従業員 WHERE 部門番号 = 1;

SELECT 氏名 FROM 営業部員;    -- 佐藤一郎，田中次郎
```

Topic 4

今回の演習と小テストの「型」

今回の小テストの「型」（小テスト2：11/25）

- **型G：SELECT文（続き）**　結合と集約を使う問合せを書く
  - 例）部門名が営業部である従業員の氏名
    → `SELECT 氏名 FROM 従業員 JOIN 部門 ON 従業員.部門番号 = 部門.部門番号 WHERE 部門名 = '営業部';`
  - 例）部門ごとの従業員数 → `SELECT 部門番号, COUNT(*) FROM 従業員 GROUP BY 部門番号;`
  - 手順：欲しい属性がどのテーブルにあるか → 2つにまたがれば JOIN ... ON（主キー＝外部キー）.「〜ごと」なら GROUP BY
- **型H：SQLとリレーショナル代数の対応**　SQL → 代数式，代数式 → SQL
  - 例）`SELECT 氏名 FROM 従業員 WHERE 職級 = 1;` → $\pi_{\text{氏名}}(\sigma_{\text{職級}=1}(\text{従業員}))$
  - WHERE が σ（内側），SELECT が π（外側），JOIN ... ON が $\bowtie$

第5回演習と次回

- ポイント資料の裏面を，表面の例にならって自分の手で解く
  - 演習1：型Gの類題（5題．sample 3題＋Chinook 2題．**実行して行数を書く**）
  - 演習2：型Hの類題（SQL → 代数 2題，代数 → SQL 2題）　確認問題：COUNT の違い，LIKE と ORDER BY
- 提出：表面・裏面を撮影して WebClass「第5回 演習」へ．締切：**次回（11/4）の講義開始30分後**
- 小テスト2（11/25）の範囲は第4〜7回．第4・5回分は型F・G・H
- 次回の予習：教科書 第6章 p.63〜72（リレーショナルデータベース設計：ER図）

小テスト1（これから15分）

- 範囲：第1〜3回．満点10点．型A〜E（第1〜3回の演習と同じ型）
- 机の上に出してよいもの：**第1〜3回のポイント資料（3枚）** と筆記用具だけ
  - ノート・教科書・スマートフォンはしまう．第4回のポイント資料もしまう
- 問題用紙と解答用紙を配る．**解答はすべて解答用紙に書く**（問題用紙にも学籍番号を書く）
- 終了の合図で筆記をやめ，問題用紙と解答用紙の両方を提出する
- 結果は次回（11/4）に返却．7点未満の人は WebClass の「小テスト1 レポート」を提出する（締切 11/11 講義開始30分後．締切後の提出は満点6点）